

Instructions

6 exercises — Promise states se lekar advanced combinators tak.

★ = interview mein aata hai — output mentally trace karo.

Console mein verify karo — har exercise ko ek .js file mein run karo.

Exercise 1 ★ — Promise State & Chaining Output

Concept: Promise States / .then() / .catch() **Task:** EXACT output order predict karo — har line explain karo:

```
console.log('1: sync start');
const p = new Promise((resolve, reject) => {
  console.log('2: executor (sync!)');
  resolve('success');
  console.log('3: after resolve (still runs!)');
});
p.then(val => {
  console.log('5: then1 ->', val);
  return 'chained';
})
.then(val => console.log('6: then2 ->', val))
.catch(err => console.log('never: catch'));
console.log('4: sync end');
// Q1: What is the exact output order?
// Q2: Why does '3' run even after resolve()?
// Q3: Why does 'then1' print 5th, not right after resolve()?
// Q4: What does 'catch' print? (trick!)
```

Key Insight / Answer

Output: 1 → 2 → 3 → 4 → 5: then1 -> success → 6: then2 -> chained

catch never prints (promise resolved, not rejected).

Q2: resolve() settles state but doesn't stop executor — remaining code runs.

Q3: .then() callbacks are MICROTASKS — run after ALL sync code completes.

Q4: catch never runs — only when rejected or error thrown.

Exercise 2 ★ — Promise Chain Error Flow

Concept: [Error Handling in Chains](#) **Task:** Har chain ka FULL output predict karo — konse .then() skip hoge, konse challenge:

```
// Chain A
Promise.resolve('start')
  .then(v => { console.log('A1:', v); return v + '->2'; })
  .then(v => { throw new Error('A broke!'); })
  .then(v => console.log('A3:', v)) // ?
  .catch(e => { console.log('A catch:', e.message); return 'recovered'; })
  .then(v => console.log('A4:', v)) // ?

// Chain B
Promise.reject(new Error('B failed'))
  .then(v => console.log('B1:', v)) // ?
  .catch(e => { console.log('B catch:', e.message); })
  .then(v => console.log('B2:', v)) // ?

// Chain C — .then(onF, onR) vs .catch()
Promise.reject(new Error('C error'))
  .then(
    v => console.log('C success'), // ?
    err => { console.log('C onRej:', err.message); return 'C fixed'; }
  )
  .then(v => console.log('C next:', v)); // ?
```

Key Insight / Answer

Chain A: A1: start → A catch: A broke! → A4: recovered. (A3 skipped)

Chain B: B catch: B failed → B2: undefined. (B1 skipped, catch returns undefined)

Chain C: C onRej: C error → C next: C fixed. (success skipped, onRej recovers)

Key: .catch() returns fulfilled promise → chain CONTINUES. Not an end.

Exercise 3 ★ — Promise.all vs allSettled

Concept: Promise Combinators **Task:** 3 APIs fetch karne hain. Dono approaches run karo aur results compare karo:

```
// Simulate APIs — p2 fails!
const fetchUser = () => new Promise(res => setTimeout(() => res({ id:1, name:'Rahul' }),
300));
const fetchPosts = () => new Promise( (_, rej) => setTimeout(() => rej(new Error('Posts API
down')), 150));
const fetchProfile = () => new Promise(res => setTimeout(() => res({ bio:'Developer' }),
200));

// Approach 1: Promise.all
console.log('--- Promise.all ---');
Promise.all([fetchUser(), fetchPosts(), fetchProfile()])
.then(results => console.log('All success:', results))
.catch(err => console.log('All failed:', err.message));
// Output? (hint: one fails)

// Approach 2: Promise.allSettled
console.log('--- Promise.allSettled ---');
Promise.allSettled([fetchUser(), fetchPosts(), fetchProfile()])
.then(results => {
  results.forEach((r, i) => {
    if (r.status === 'fulfilled')
      console.log(`API ${i+1} success:`, r.value);
    else
      console.log(`API ${i+1} failed:`, r.reason.message);
  });
});

// Q: Which approach is better for a dashboard
// where partial data is OK?
```

Key Insight / Answer

Promise.all output: 'All failed: Posts API down' at ~150ms. User+Profile data lost!

allSettled output: API 1 success, API 2 failed, API 3 success — all data available.

Dashboard ke liye: allSettled — partial data better than no data.

allSettled timing: ~300ms (waits for all). Promise.all: ~150ms (fails fast).

Exercise 4 — Retry with Delay (Real-world Pattern)

Concept: Promisification + Chaining **Task:** Generic `retryWithDelay(fn, retries, delayMs)` function implement karo — failed promises ko exponential backoff ke saath retry kare:

```
const delay = ms => new Promise(res => setTimeout(res, ms));

function retryWithDelay(fn, maxRetries, baseDelay) {
  return new Promise((resolve, reject) => {
    function attempt(retriesLeft, attemptNum) {
      fn()
        .then(resolve) // success — done!
        .catch(err => {
          if (retriesLeft === 0) {
            reject(new Error(`Failed after ${maxRetries} attempts: ${err.message}`));
            return;
          }
          const wait = baseDelay * Math.pow(2, attemptNum); // exponential
          console.log(`Attempt ${attemptNum+1} failed. Retrying in ${wait}ms...`);
          delay(wait).then(() => attempt(retriesLeft - 1, attemptNum + 1));
        });
    }
    attempt(maxRetries, 0);
  });
}

// Test: flaky API that fails 70% of the time
let callCount = 0;
function flakyApi() {
  callCount++;
  console.log(`API called (attempt ${callCount})`);
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (Math.random() < 0.7) reject(new Error('Server error'));
      else resolve({ data: 'success!' });
    }, 100);
  });
}

retryWithDelay(flakyApi, 3, 100)
  .then(data => console.log('Finally got:', data))
  .catch(err => console.log('Gave up:', err.message));

// Retries with: 100ms, 200ms, 400ms gaps
```

Key Insight / Answer

Exponential backoff: 100ms, 200ms, 400ms — each retry waits double.

`fn()` returns new Promise each attempt — not the same promise retried.

`resolve/reject` captured in outer Promise — classic 'deferred' pattern.

Production: add jitter (random extra ms) to avoid 'thundering herd' problem.

Exercise 5 — Promise.race for API Timeout

Concept: `Promise.race()` **Task:** `Promise.race()` use karke har API call pe 2-second timeout implement karo — timeout hone par meaningful error do:

```
function withTimeout(promise, ms, label = 'Operation') {
  const timeoutP = new Promise((_, reject) =>
    setTimeout(() => reject(new Error(`${label} timed out after ${ms}ms`)),
      ms)
  );
  return Promise.race([promise, timeoutP]);
}

// Simulate slow API
function slowFetch(url, simulatedDelay) {
  return new Promise(resolve =>
    setTimeout(() => resolve({ url, data: 'response!' }), simulatedDelay)
  );
}

// Fast API — succeeds
withTimeout(slowFetch('/api/users', 1000), 2000, 'Users API')
  .then(res => console.log('Users:', res.data))
  .catch(err => console.log('Users error:', err.message));

// Slow API — times out
withTimeout(slowFetch('/api/products', 3000), 2000, 'Products API')
  .then(res => console.log('Products:', res.data))
  .catch(err => console.log('Products error:', err.message));

// Q: What prints for each? When?
// Q: What happens to the slow fetch Promise after timeout?
// (Hint: Promises can't be cancelled in standard JS)
```

Key Insight / Answer

Users: 'Users: response!' at ~1000ms (fast API wins race).

Products: 'Products error: Products API timed out after 2000ms' at 2000ms.

After timeout: `slowFetch(3000)` Promise still runs in background! Resolves at 3s.

No cancellation in standard JS — `AbortController` for fetch API only.

Exercise 6 — Build: Promisify from Scratch

Concept: Promisification **Task:** Generic `promisify()` aur `promisifyAll()` banao, phir NodeJS-style callback functions ko convert karo:

```
// Generic promisify
function promisify(fn) {
  return function(...args) {
    return new Promise((resolve, reject) => {
      fn.call(this, ...args, (err, ...results) => {
        if (err) { reject(err); return; }
        // Handle multiple results: (err, result1, result2)
        resolve(results.length <= 1 ? results[0] : results);
      });
    });
  };
}

// promisifyAll – convert all methods of an object
function promisifyAll(obj) {
  const result = {};
  for (const [key, val] of Object.entries(obj)) {
    if (typeof val === 'function') {
      result[key] = promisify(val.bind(obj));
    }
  }
  return result;
}

// Test with fake callback APIs
const callbackDb = {
  findUser(id, cb) {
    setTimeout(() => {
      if (id > 0) cb(null, { id, name: `User${id}` });
      else cb(new Error('Invalid ID'));
    }, 100);
  },
  saveUser(user, cb) {
    setTimeout(() => cb(null, { ...user, saved: true }), 50);
  },
};

const promiseDb = promisifyAll(callbackDb);

promiseDb.findUser(1)
  .then(user => { console.log('Found:', user); return promiseDb.saveUser(user); })
  .then(saved => console.log('Saved:', saved))
  .catch(err => console.log('Error:', err.message));

promiseDb.findUser(-1)
  .catch(err => console.log('Find failed:', err.message));
```

Key Insight / Answer

`fn.call(this, ...)` — method context preserve karta hai (important for object methods).

Multiple results: `(err, result1, result2) -> resolve([result1, result2])`.

`promisifyAll`: entire API object convert — production mein use hota hai (e.g. bluebird library).

`util.promisify` in Node.js yahi karta hai internally.